

Laboratorio 2 — ¿La memoria importa?

Alumno: Anthony Angel Briceño Quiroz

Curso: COMPUTACIÓN PARALELA Y DISTRIBUIDA

- Parte A - Modelo de Von Neumann:

- 1) Explique brevemente el modelo Von Neumann e identifique CPU, memoria, unidad de control y unidad aritmético lógica:

Esta arquitectura es una de las estructuras que ha servido como cimiento para todos los tipos de computadoras, desde una PC hasta un celular, consolas. Fue presentada en el año 1945 y es común verla en todos los procesadores. Ésta se basa en el concepto de “Programa almacenado”, donde los datos de la instrucción y los datos del programa se almacenan en la misma memoria.

- CPU: Es la Unidad Central de Procesamiento y se divide en dos partes
 - Unidad de Control: Encargado de poder decidir que instrucciones de un programa se debe ejecutar (Jefe).
 - Unidad Aritmética y Lógica: Responsable de ejecutar la instrucción actual (Trabajador).
- Memoria: Es el conjunto de posiciones en donde se puede almacenar tanto instrucciones como datos.

- 2) ¿Por qué el acceso a memoria puede convertirse en un factor que limite el rendimiento del procesador?

- Porque como sabemos la arquitectura actual que se usa tiene ciertos problemas, el cuello de botella que existe llega a ser uno de ellos, la CPU y la Memoria están separados por lo que la conexión entre estos elementos llega a ser mediante el bus el cual puede llegar a tener una capacidad física limitada de transmisión de datos. Por otro lado creo que también se relaciona con el ancho de banda, fallos en la caché y la memoria virtual y los fallos de las páginas.

- 3) ¿Por qué se utilizan diferentes niveles de memoria entre el procesador y la memoria principal?

- Porque estos niveles permiten poder hacer consultas de manera mas rápida y no ir y hacer consultas directo a la memoria; L1, L2 y L3; donde L1 llega a ser el nivel mas pequeño y mas rápido como tal y L3 el mas grande pero a la vez más lento. Así mismo los programas siguen el principio de localidad, el cual se divide en dos partes: Localidad temporal y espacial, osea si un procesador accede a una variable o instrucción es muy probable que se vuelva a acceder otra vez a dicha variable o instrucción y la espacial en la cual si un procesador accede a una dirección de memoria es probable que acceda a una dirección cercana a la que se accedió anteriormente. Y otro punto es el tema de Cache Hit y el Cache Miss, osea va buscando y se empieza en L1, si no está pasa a L2 y luego pasa a L3 si no se encuentra en ninguno de lo Ln, se produce un Cache Miss que obliga al sistema a hacer una consulta lenta directamente a la memoria principal.

- Parte B - Acceso a una matriz

Trabaje con una matriz cuadrada de números reales. Se recomienda comenzar con $N = 4000$, ajustándolo según las características del equipo. Inicialice la matriz antes de realizar las mediciones y registre las características básicas del equipo.

Antes de empezar a hacer pruebas como tal, lo que hice fue crear un archivo llamado [mainMatriz.cpp](#), el cual lo creé con la finalidad de poder obtener información con respecto a la ram, en este caso en dicho código creo una matriz como un array de arrays, de esta manera se va al stack dicha matriz, justo por eso en esta caso el tamaño del stack importa, en mi pc dicho tamaño es de 8MB por lo que si lo analizamos en base a los elementos de la matriz que son doubles, serian 8 bytes cada double que en resumen serian 1 048 576 elementos totales donde su raíz es 1024, osea el N máximo para mi stack seria 1024, osea 1000 en términos de las pruebas que tendríamos que hacer. Por lo que opte por usar un vector unidimensional para evitar errores humanos en este caso referentes a poder limpiar memoria con delete, por lo que pase a crear el archivo [mainLab2.cpp](#), el cual se indexa a mano $A[i*N+j]$, cabe aclarar que uso un vector unidimensional porque si creaba un vector de vector de doubles tendria memoria dispersa y no junta y a mi punto de vista eso afecta con respecto a acceder a filas y a columnas, y obvio los resultados serian distintos.

```

1  #include <iostream>
2  #include <chrono>
3  #include <vector>
4  #include <cstdlib>
5
6  using namespace std;
7
8  int main(int argc, char* argv[]){
9      if (argc < 2){
10         cerr << "Uso: " << argv[0] << " <N>\n";
11         return 1;
12     }
13     int N = atoi(argv[1]);
14
15     // Solo la columna 0 de una matriz N x N importa para este experimento;
16     // usar un vector de tamaño N evita reservar N*N memoria innecesaria.
17     // esta variable es volatile esto quiere decir que el compilador no puede eliminar
18     // las escrituras de él
19     volatile double sumidero = 0.0;
20
21     // Patrón 1: reutiliza A[i][0] tres veces seguidas (localidad temporal alta)
22     vector<double> A1(N, 0.0);
23     auto t1 = chrono::steady_clock::now();
24     for (int i = 0; i < N; i++){
25         A1[i] += 1;
26         A1[i] += 1;
27         A1[i] += 1;
28     }
29     auto t2 = chrono::steady_clock::now();
30     auto tiempoReutilizado = chrono::duration_cast<chrono::microseconds>(t2 - t1);
31     for (int i = 0; i < N; i++) sumidero += A1[i];
32
33     // Patrón 2: tres pasadas completas separadas sobre la misma columna (localidad temporal baja)
34     vector<double> A2(N, 0.0);
35     auto t3 = chrono::steady_clock::now();
36     for (int k = 0; k < 3; k++){
37         for (int i = 0; i < N; i++){
38             A2[i] += 1;
39         }
40     }
41     auto t4 = chrono::steady_clock::now();
42     auto tiempoSeparado = chrono::duration_cast<chrono::microseconds>(t4 - t3);
43     for (int i = 0; i < N; i++) sumidero += A2[i];
44
45     cout << "N," << N
46         << ",reutilizado_us," << tiempoReutilizado.count()
47         << ",separado_us," << tiempoSeparado.count() << "\n";
48
49     return 0;
50 }

```

- **Parte C - Recorrido por filas**

Mida el tiempo de ejecución y realice varias ejecuciones

Para esta tarea como tal, antes de poder recorrer las filas inicializamos la matriz, con este formato $i * 0.5 * j$, esto se hizo antes de cronometrar el tiempo. Como tal para acceder a los datos, como tenemos un vector unidimensional lo que hacemos es acceder mediante: $A[i*N+j]$, esto permite simular una posición fila columna, para luego tomar el tiempo de acceso con chronos y bueno en este caso cuando se accede por filas, sabemos que el CPU no trae solo un elemento osea por ejemplo $A[0]$, si no que trae un bloque de 64 bytes (es una línea de cache típica), esto quiere decir que cuando se accede a $A[0]$ se trae también a $A[1] \dots A[7]$, y cuando busque a uno de ellos ya estarán ahí y habra un cache hits. Osea solo cada 8 accesos hay que ir a buscar a un bloque nuevo. Con un $N = 4000$ eso es $4000 \times 4000 / 8 = 2\,000\,000$ millones de “viajes” a memoria en ves de 16 millones.

- **Parte D - Recorrido por columnas**

Repita las mediciones y verifique que ambas versiones realizan esencialmente el mismo trabajo.

Ahora cuando hacemos un recorrido por columnas ahí está el detalle, lo que pasa es que como ya sabemos la CPU trae de 8 en 8 bloques, pero como hacemos un recorrido por columnas no accedemos ni a $A[1]$, ni a $A[2] \dots A[7]$, si no que accedemos a $A[40000]$ osea lo que trajimos nunca fue accedido en ese instante de cambio de columna, entonces cada acceso individual hace que se fuerze a un viaje nuevo a la memoria, osea como no encontramos lo que buscamos se crea un cache miss, por lo que Con $N = 4000$ son algo de 16 millones de viajes reales.

- **Parte E - Comparación experimental**

Realice el experimento para diferentes tamaños de matriz.

Tamaño N	Filas (ms)	Columnas (ms)	Relación
500	0.077 ms	0.146 ms	1.896103896103896
1000	0.2845 ms	0.584 ms	2.0527240773286466
2000	1.359 ms	3.6015 ms	2.6501103752759385
4000	5.678 ms	30.8165 ms	5.4273511799929555

Calcule: $R = T_{\text{columnas}} / T_{\text{filas}}$

- **Parte F - Localidad espacial**

1) ¿Cuál de los dos recorridos aprovecha mejor la localidad espacial?

En este caso el acceso por filas es el que aprovecha mejor la localidad espacial, dicho principio menciona que si un elemento es accedido, es muy probable que un elemento vecino de dicho elemento accedido sea accedido después.

- 2) ¿Por qué los accesos $A[i][0]$, $A[i][1]$, $A[i][2]$, ... pueden comportarse diferente de $A[0][j]$, $A[1][j]$, $A[2][j]$,...?

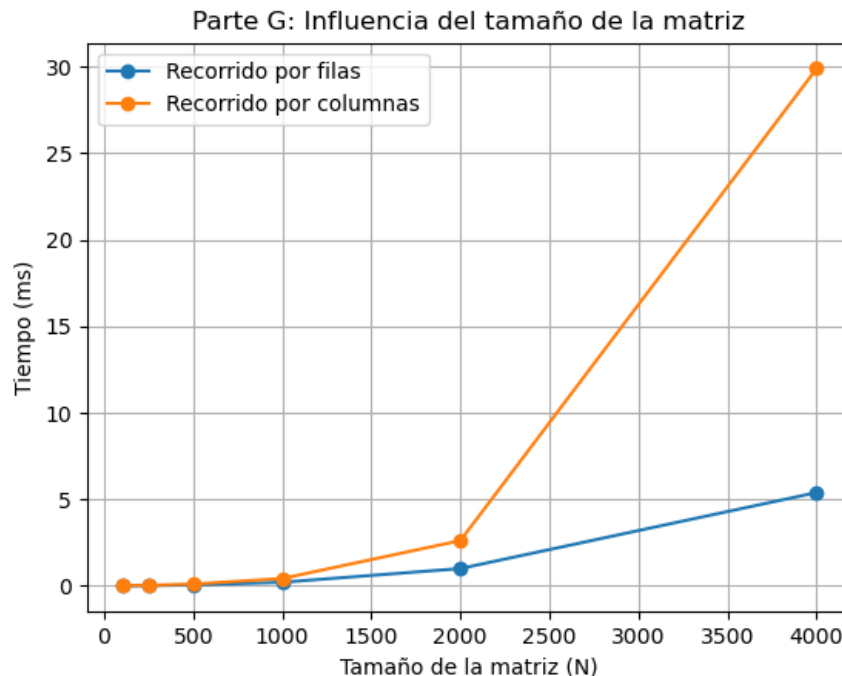
Ok, bueno el primer acceso es el acceso por filas o sea que mantenemos ese $i = 0$ en un buen tiempo en este caso como se aprovecha la localidad espacial y el tema de traer elementos consecutivos, esto permite poder acceder de manera mas rápida a los elementos, sin embargo con el acceso de columnas estamos cambiando la columna a la que accedemos o sea debemos pasar por ejemplo del elemento $A[0][1]$ al elemento $A[1][1]$, pero esto si se ve influenciado por el tamaño de N , porque si tenemos un N mas grande tendremos que dar un salto mucho mas grande, así mismo el tema de poder obtener acceso rápido a elementos consecutivos ya no se cuenta acá porque saltamos de columna por columna.

- 3) ¿Cómo puede relacionarse esta diferencia con la memoria caché?

Es facil de entender como sabemos la cache trae por bloques pero dichos bloques son de elementos consecutivos, por lo que el acceso por filas es muy muy rápido pero el acceso por columnas es lento lento por lo mismo que la cache usa el principio de localidad espacial o sea si traigo $A[0]$ puedo traer a $A[7]$, porque es probable que se acceda luego a ese elemento.

- **Parte G - Influencia del tamaño del problema:**

Construya una gráfica que compare el tiempo de ambos patrones en función de N .



- **Parte H - Localidad temporal:**

Compare los siguientes patrones:

[localidadTemporal.cpp](#)

```
1  for (int i = 0; i < N; i++){
2      A1[i] += 1;
3      A1[i] += 1;
4      A1[i] += 1;
5  }
```

```

1  for (int k = 0; k < 3; k++){
2      for (int i = 0; i < N; i++){
3          A2[i] += 1;
4      }
5  }

```

```

1  #include <iostream>
2  #include <chrono>
3  #include <vector>
4  #include <cstdlib>
5
6  using namespace std;
7
8  int main(int argc, char* argv[]){
9      if (argc < 2){
10         cerr << "Uso: " << argv[0] << " <N>\n";
11         return 1;
12     }
13     int N = atoi(argv[1]);
14
15     // Solo la columna 0 de una matriz N x N importa para este experimento;
16     // usar un vector de tamaño N evita reservar N*N memoria innecesaria.
17     // esta variable es volatile esto quiere decir que el compilador no puede eliminar
18     // las escrituras de él
19     volatile double sumidero = 0.0;
20
21     // Patrón 1: reutiliza A[i][0] tres veces seguidas (localidad temporal alta)
22     vector<double> A1(N, 0.0);
23     auto t1 = chrono::steady_clock::now();
24     for (int i = 0; i < N; i++){
25         A1[i] += 1;
26         A1[i] += 1;
27         A1[i] += 1;
28     }
29     auto t2 = chrono::steady_clock::now();
30     auto tiempoReutilizado = chrono::duration_cast<chrono::microseconds>(t2 - t1);
31     for (int i = 0; i < N; i++) sumidero += A1[i];
32
33     // Patrón 2: tres pasadas completas separadas sobre la misma columna (localidad temporal baja)
34     vector<double> A2(N, 0.0);
35     auto t3 = chrono::steady_clock::now();
36     for (int k = 0; k < 3; k++){
37         for (int i = 0; i < N; i++){
38             A2[i] += 1;
39         }
40     }
41     auto t4 = chrono::steady_clock::now();
42     auto tiempoSeparado = chrono::duration_cast<chrono::microseconds>(t4 - t3);
43     for (int i = 0; i < N; i++) sumidero += A2[i];
44
45     cout << "N," << N
46          << ",reutilizado_us," << tiempoReutilizado.count()
47          << ",separado_us," << tiempoSeparado.count() << "\n";
48
49     return 0;
50 }
51

```

- 1) ¿Existe reutilización de los mismos datos?

Si, porque en el patrón 1 se ve explícitamente que el elemento de A1 osea el [i] se USA 3 VECES, mejor dicho se accede a la celda 3 veces, en este caso la diferencia no es CUANTO si no CUANDO y como vemos se ve que se usa inmediatamente después de la misma instrucción, osea sumarle 1 al elemento. Por

otro lado en el patrón 2 se ve pero se hace recorriendo el arreglo entre cada suma a $A[i]$ se tiene que recorrer todo el arreglo de N elementos.

- 2) ¿Qué relación tiene este experimento con la localidad temporal?

Lo que dice el principio de localidad temporal es que si se accede a un dato, es muy probable que se acceda en un futuro cercano, y como vemos en ambos patrones se accede, en uno mas antes que otro pero se accede, eso suele ocurrir comúnmente en bucles, no en todos los bucles de un programa pero si en la mayoría.

- 3) ¿Los resultados experimentales coinciden necesariamente con lo esperado? Explique.

Con $N = 1\,000\,000$ si coincidió con lo esperado, esto pasa porque el vector no cabe en la caché, así que la pérdida de localidad temporal es real, pero N sería más pequeño, el vector cabría en $L1$, y bueno cada pasada que se haga se encontraría rápidamente y los tiempos entre ambos patrones sería probablemente casi iguales. Osea el principio de localidad temporal NO es absoluto, depende del conjunto de datos a los que se acceden entre repeticiones, osea si es más grande o más chico; Es una relación entre el patrón de acceso y el tamaño del problema respecto a la caché, no una propiedad fija del código.

- **Parte I - Características de la memoria caché:**

Obtenga información del procesador del equipo utilizado y registre los tamaños disponibles de $L1$, $L2$ y $L3$

Características	Valor
Procesador	AMD Ryzen 9 8945HX
Número de núcleos	16 núcleos físicos (32 hilos, 2 por núcleo)
$L1d$	512 KB (16 instancias)
$L1i$	512 KB (16 instancias)
$L2$	16 MB (16 instancias)
$L3$	64 MB (2 instancias)
Memoria RAM	32 GB
Sistema Operativo	Ubuntu 24.04

- **Parte J - Análisis y Preguntas Finales:**

- 1) Relacione los resultados obtenidos con el modelo de Von Neumann.

Este modelo separa el CPU de la Memoria, lo que une a estos elementos es el bus con capacidad LIMITADA, el famoso CUELLO DE BOTELLA, la CPU podía ejecutar las multiplicaciones casi instantáneas, pero el tiempo real termina dominado por cuánto tardaba en TRAER los datos desde memoria a través de ese bus. Filas vs columnas no cambia el trabajo de la ALU cambia cuánto tráfico por el bus (y cuántos viajes largos a RAM) se generan.

- 2) ¿Por qué una CPU rápida puede permanecer esperando datos de memoria?

Porque la velocidad del procesador (ciclos de reloj, GHz) creció mucho más rápido históricamente que la velocidad de la RAM. Un cache miss cuesta ~ 100 -300 ciclos de espera durante ese tiempo, la CPU (aunque sea capaz de miles de millones de operaciones por segundo) está parada, sin nada que procesar, esperando el dato. El $R \approx 5.4$ en $N=4000$ es literalmente eso: mismo trabajo de ALU, pero mucho más tiempo total porque la CPU esperó memoria en vez de calcular.

- 3) ¿Qué problema intenta resolver la memoria caché?

Resuelve exactamente esa brecha de velocidad (CPU rápida, RAM lenta) interponiendo niveles intermedios ($L1/L2/L3$) cada vez más rápidos y más chicos, que guardan copias de los datos usados recientemente o cercanos a los usados recientemente apostando a que el programa va a reutilizarlos pronto (localidad temporal) o va a necesitar los vecinos (localidad espacial). Así, la mayoría de los accesos se resuelven rápido sin tener que ir hasta RAM cada vez.

- 4) ¿Qué relación existe entre memoria caché, localidad, patrón de acceso y tiempo de ejecución?

Es una cadena causal: el patrón de acceso que tú codificas (orden de tus loops) determina cuánta localidad (espacial y/o temporal) explota tu programa -> eso determina cuántos hits vs misses produce en la caché -> eso determina cuántas veces hay que ir a niveles lentos de memoria -> eso determina el tiempo de ejecución final. Los dos experimentos (E/G con filas vs columnas, y H con reutilizado vs separado) son ejemplos de esta misma cadena, cada uno probando un tipo de localidad distinto.

- 5) ¿Por qué dos programas que realizan esencialmente las mismas operaciones pueden presentar tiempos diferentes?

Porque el número de operaciones aritméticas no es el único factor de rendimiento real. El PATRÓN de acceso a memoria importa igual o más dos programas con la misma cantidad de multiplicaciones dieron tiempos muy distintos solo por el orden de acceso.

- 6) ¿Qué diferencia existe entre localidad espacial y localidad temporal?

La localidad espacial se centra principalmente en bloques de datos o sea si accedí al A[0] hay probabilidad de acceder a A[7], la localidad temporal se basa en un dato como tal o sea ya accedí a A[0], ok perfecto si yo a ese elemento le hago algo o lo manipulo lo debería guardar de alguna manera porque puedo accederlo luego, un ejemplo mas claro es si tengo una variable suma que constantemente estoy que la actualizo con respuestas de la suma de elementos entonces si ya accedí una vez a esa variable probablemente pueda accederse nuevamente.

Conclusiones:

Con este laboratorio pude comprobar de manera experimental que el rendimiento de un programa no depende únicamente de cuántas operaciones realiza, sino también de cómo accede a la memoria. En mis pruebas, el recorrido por filas y el recorrido por columnas realizaron exactamente el mismo número de operaciones (N^2 multiplicaciones), con el mismo código de acceso ($A[i*N+j] *= 2.0$), y aun así los tiempos fueron muy distintos, especialmente conforme aumentaba el tamaño de la matriz. Con $N = 4000$ la relación R llegó a ser de aproximadamente 5.4, es decir, el recorrido por columnas tardó más de 5 veces lo que tardó el recorrido por filas, solamente por el orden en que se visitaron las celdas. Esto se relaciona directamente con la jerarquía de memoria caché (L1, L2, L3) que investigué en la Parte I. Mientras la matriz es pequeña y cabe dentro de algún nivel de caché, la diferencia entre ambos recorridos es más moderada, pero cuando el tamaño de la matriz supera la capacidad de la caché (en mi caso, L3 de 32MB, y mi matriz de $N=4000$ ocupa 128MB), el patrón de acceso que no aprovecha la localidad espacial empieza a generar muchos más cache misses, obligando a la CPU a esperar constantemente datos que vienen de la memoria RAM, mucho más lenta que la caché. Con el experimento de localidad temporal (Parte H) confirmé algo parecido: reutilizar un mismo dato varias veces seguidas es mucho más rápido que acceder a él, alejarse, y volver más tarde, porque en el segundo caso el dato ya fue desplazado de la caché. En conclusión, la memoria caché tiene una influencia enorme en el rendimiento real de un programa, incluso cuando la cantidad de trabajo matemático es idéntica. Por eso, al programar, no basta con pensar en la complejidad algorítmica (cuántas operaciones se hacen), también hay que pensar en el patrón de acceso a memoria, ya que este puede ser la diferencia entre un programa rápido y uno mucho más lento.